

수업 05 · 2026-10-20

AI 코드에이전트

AI에 코드 작성을 요청하고 답변을 검토합니다. 바뀐 코드와 실행 결과를 확인하며 잘못된 답변을 찾아봅니다.

10:00~17:00 · 50분 수업 후 10분 휴식 · 중식 11:50~13:00

오늘 필수 활동 10개 · 교시별 설명과 예제를 보고 직접 해 봅시다.

처음 시작하기 · AI 코드에이전트

1. ZIP을 일반 폴더에 풀고 examples/course의 data·work·code 위치를 유지합니다. 배포 ZIP은 course 폴더부터 열 수 있습니다.
2. [교안](#)에서 개념을 읽고 [실습지](#)의 A01부터 진행합니다.
3. work 파일만 수정하고 원본은 보존합니다. 같은 파일을 저장·다시 열어 확인합니다.
4. 개인 실행을 못 했으면 예상과 강사 실행 결과를 대조하고 확인 방법을 정확히 적습니다.

실행 준비

브라우저와 텍스트 편집기가 기본입니다. Python 명령의 현재 폴더는 압축을 푼 course입니다. Windows에서 python 대신 py만 있으면 강사의 확인 아래 py -3로 대체합니다.

```
python code/check_cases.py code/baseline.py
python code/check_cases.py code/proposal_a.py
python code/check_cases.py code/reviewed.py
python code/check_cases.py work/classifier.py
```

실행하지 못하면 파일 경로·명령·오류 문구를 기록해 도움을 요청합니다. 코드 상자를 보는 것은 Python 실행이 아닙니다. 실제 AI 계정·모델·유료 API를 호출하지 않습니다. 제공 응답은 강사가 만든 비교 자료입니다. 합성 분류 규칙이며 실제 센서 안전 판단을 검증하지 않았습니다.

수업 전, 이 세 가지부터 확인하세요

앞선 내용이 기억나지 않으면 아래 질문부터 읽어 보세요. 먼저 답을 적고 풀이와 비교하면 됩니다.

요청과 정답

질문: AI가 실행할 수 있는 코드를 내놓으면 업무 요구를 만족하나요?

풀이: 실행 성공과 요구 충족은 다릅니다. 구체적인 입력, 규칙, 기대 결과로 비교해야 합니다.

근거

질문: AI가 모두 테스트했다고 말하면 확인을 끝내도 될까요?

풀이: 실제 실행 명령, 입력, 출력과 실패 사례를 확인합니다. 수행하지 않은 검증은 별도로 남깁니다.

범위

질문: 한 함수 수정을 맡겼는데 전체 파일이 바뀌면 무엇을 보나요?

풀이: 변경 차이를 읽고 요청 범위와 비교합니다. 필요 없는 변경을 분리한 뒤 정상·경계·누락 입력을 확인합니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02, A03	새 입력으로 한 번 더 확인
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11, A12	새 입력으로 한 번 더 확인
5	A13, A14, A15	새 입력으로 한 번 더 확인
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 사례와 풀이를 읽고, 이번 교시에 정한 활동을 직접 해 봅니다. 활동 결과를 확인한 뒤에는 추가 문제를 풀어 봅니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

이전 Python·Git 기초를 짧게 복습하며 준비된 작은 예제를 읽고 수정합니다. **최신 운영은 50분 × 6교시, 순수 수업 300분**입니다. 설명 보충은 필수 개념이며 독립 과제는 추가 복습입니다. 추가 문제는 수업 후 복습에 사용합니다.

실제 AI 계정·모델·유료 API를 호출하지 않습니다. 제공 응답은 강사가 만든 비교 자료입니다. 합성 분류 규칙이며 실제 센서 안전 판단을 검증하지 않았습니다.

전체 코드와 모든 줄의 뜻·실행 방법은 [코드 해설](#)에 있습니다. 작성 소스와 생성된 번들·잠금 파일을 구분합니다.

1교시 · 작업을 맡기는 방법

10:00~10:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 품니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A01 · 에이전트와 사람의 역할

사례로 이해하기 · 실행하는 AI와 책임을 갖는 사람

실무 상황

센서 안내 프로그램을 고치려는 직원이 AI에게 “온도 분류를 개선해 줘”라고 보냅니다. AI는 파일 이름과 기준을 추측하고 새 코드를 제안할 수 있습니다. 직원이 기대한 것은 경계 오류 하나의 수정인데 프로그램 전체가 바뀌면 검토가 어려워집니다. 먼저 말길 범위와 사람이 결정할 규칙을 구별합니다.

쉬운 풀이

대화형 AI는 문장이나 코드를 답할 수 있습니다. 코드에이전트는 허용된 도구를 이용하여 파일을 읽고 고치거나 실행까지 이어갈 수 있습니다. 가능한 능력과 이번 작업에서 허용한 행동은 다릅니다. 프롬프트는 요청 글이고 컨텍스트는 그 요청을 이해하는 데 필요한 자료입니다. 사람은 업무 기준, 공유할 자료, 적용 승인, 실제 확인 범위를 결정합니다. 도구가 읽지 못한 숨은 규칙까지 안다고 추정하지 않습니다.

완성 예와 풀이

요청의 목적을 "28 이상을 주의로 분류"로 적습니다. 대상은 work/classifier.py의 classify 함수이며 입력은 가상 온도, 출력은 정상·주의·미측정·입력 오류입니다. 관련 자료는 현재 함수와 cases.json, 실패 출력 마지막 부분입니다. 읽기와 수정은 이 연습 복사본에만 허용하고 외부 게시와 계정 접근은 허용하지 않습니다. 실제 토큰 대신 DEMO_TOKEN이라는 표시를 씁니다. 확인 근거는 변경 비교와 같은 검사 입력의 실행 결과입니다. 제공 AI 응답은 수업용 작성 자료이며 실제로 모델을 호출해서 받은 결과가 아닙니다.

독립 추가 복습 과제

파일 세 개, 고객 명단, 접속 암호, 가상 실패 로그 가운데 온도 경계 수정에 필요한 자료를 선택하세요. 이어서 "목적 / 바꿀 곳 / 유지할 것 / 금지할 행동 / 확인 근거" 다섯 줄 요청서를 작성하세요.

답과 대표 오답

현재 함수·검사 입력·가상 실패 로그를 선택하고 고객 명단과 암호는 제외합니다. "자료를 전부 보내면 정확해진다"는 답은 관련성과 공유 범위를 검토하지 않았습니다. "AI가 썼으니 실행도 했다" 역시 생성과 실행의 차이를 놓칩니다.

확인 질문과 해설

① 에이전트 이름이 능력을 보장하나요? 실제 연결 도구와 권한을 확인해야 합니다. ② 비밀이 필요한 상황은 어떻게 하나요? 그대로 공유하지 말고 담당 절차와 대체 데이터를 확인합니다. ③ 완료 판단은 누구의 일인가요? 사람이 요청한 조건과 바뀐 코드, 실행 결과를 직접 비교해야 합니다.

뜻과 비유. 코드에이전트는 요청을 해석하고 파일과 실행 도구를 사용해 작업을 이어 가는 AI 도구입니다. 대화 답변 한 번과 파일 변경·실행이 연결된 작업 흐름을 구분합니다.

왜 필요한가. 동료에게 일을 맡겨도 완료 기준은 함께 정합니다. 코드가 생겼다는 사실과 업무 요구가 충족됐다는 사실은 다릅니다.

예를 따라 읽기. 센서 온도 28을 주의로 분류해 달라는 요청은 입력·기준·출력·확인 방법을 포함할 수 있습니다.

헛갈리는 반례. "알아서 좋게 고쳐"는 유지해야할 함수 이름이나 28 °C 경계를 알려 주지 않습니다.

직접 적용. 요청서에서 목적을 한 문장으로 찾기 → AI가 제안할 일과 사람이 판단할 일 나누기 → 고친 파일과 실행 결과 두 가지 적기 → 짝에게 도구와 검토자의 역할 설명하기

설명할 질문. 코드가 생긴 것만으로 완료가 아닌 이유는 무엇인가요?

막혔을 때. 도구 이름과 능력을 혼동하면 읽기·변경·실행 중 실제로 허용한 행동을 따로 적습니다.

입력·답변·확인 순서는 [실습지의 A01](#)과 [활동 원문](#)에 있습니다.

A01 · 직접 해보기

A01 · 에이전트와 사람의 역할

목적

동료에게 일을 맡겨도 완료 기준은 함께 정합니다. 코드가 생겼다는 사실과 업무 요구가 충족됐다는 사실은 다릅니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [request.md](#), [A01_input.md](#)

작업: [A01_notes.md](#)

실제 입력

센서 온도 28을 주의로 분류해 달라는 요청은 입력·기준·출력·확인 방법을 포함할 수 있습니다.

단계

1. 요청서에서 목적을 한 문장으로 찾기
2. AI가 제안할 일과 사람이 판단할 일 나누기
3. 고친 파일과 실행 결과 두 가지 적기
4. 짝에게 도구와 검토자의 역할 설명하기

예상 결과

목적·역할·완료 근거가 있는 작업 설명

확인 방법

코드가 생긴 것만으로 완료가 아닌 이유는 무엇인가요?

오류 해결

도구 이름과 능력을 혼동하면 읽기·변경·실행 중 실제로 허용한 행동을 따로 적습니다.

힌트

코드에이전트는 요청을 해석하고 파일과 실행 도구를 사용해 작업을 이어 가는 AI 도구입니다. 대화 답변 한 번과 파일 변경·실행이 연결된 작업 흐름을 구분합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 코드가 생긴 것만으로 완료가 아닌 이유는 무엇인가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

A02 · 작은 작업과 수정 범위

뜻과 비유. 작업 범위는 어떤 파일과 동작을 바꿀지 정한 경계입니다. 전체 프로그램을 한 번에 고치기보다 한 함수의 분류 규칙처럼 확인 가능한 단위로 나눕니다.

왜 필요한가. 범위를 알면 변경 비교가 짧아지고 잘못 바꾼 항목을 되돌리기 쉽습니다. 10/16의 변경 비교가 여기서 업무 검토 근거가 됩니다.

예를 따라 읽기. 오늘은 classifier.py의 classify 동작만 바꾸고 함수 이름과 threshold =28을 유지합니다.

헛갈리는 반례. 결과를 맞추려고 cases.json 기대 값을 변경하면 검사기준을 바꾼 것이므로 요청한 코드를 올바르게 고쳤다고 할 수 없습니다.

직접 적용. 바꿀 파일 하나 적기 → 유지할 이름·기준·입출력 찾기 → 요청 밖 변경 예 두 가지 표시하기 → 작업 요청서에 범위 적기

설명할 질문. 테스트 기대 값을 바꾸면 왜 문제가 숨을 수 있나요?

막혔을 때. 파일을 못 찾으면 압축을 푼 course 폴더의 code·data·work 위치부터 확인합니다.

입력·답칸·확인 순서는 [실습지의 A02](#)와 [활동 원문](#)에 있습니다.

A02 · 직접 해보기

A02 · 작은 작업과 수정 범위

목적

범위를 알면 변경 비교가 짧아지고 잘못 바꾼 항목을 되돌리기 쉽습니다. 10/16의 변경 비교가 여기서 업무 검토 근거가 됩니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [request.md](#), [task.md](#), [A02_input.md](#)

작업: [A02_notes.md](#), [task.md](#)

실제 입력

오늘은 classifier.py의 classify 동작만 바꾸고 함수 이름과 threshold =28을 유지합니다.

단계

1. 바꿀 파일 하나 적기
2. 유지할 이름·기준·입출력 찾기
3. 요청 밖 변경 예 두 가지 표시하기
4. 작업 요청서에 범위 적기

예상 결과

변경 가능·유지 범위가 구분된 요청서

확인 방법

테스트 기대 값을 바꾸면 왜 문제가 숨을 수 있나요?

오류 해결

파일을 못 찾으면 압축을 푼 course 폴더의 code·data·work 위치부터 확인합니다.

힌트

작업 범위는 어떤 파일과 동작을 바꿀지 정한 경계입니다. 전체 프로그램을 한 번에 고치기보다 한 함수의 분류 규칙처럼 확인 가능한 단위로 나눕니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 테스트 기대 값을 바꾸면 왜 문제가 숨을 수 있나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 컨텍스트와 관련 자료

[이 교시의 다른 예와 코드 해설 열기](#)

2교시 · 검사할 조건을 적은 요청서

11:00~11:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 품니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A05 · 정상·경계·오류 사례

뜻과 비유. 검사 사례는 입력과 기대 결과를 한 쌍으로 적은 것입니다. 경계 사례는 규칙이 달라지는 바로 앞·같은 값·바로 뒤를 비교합니다.

왜 필요한가. 평범한 22.5 하나가 통과해도 28에서 비교 기호가 틀린 문제는 드러나지 않습니다.

예를 따라 읽기. 27.9는 정상, 28 과 28.1은 주의입니다. 0은 유효하며 None은 미측정입니다.

헛갈리는 반례. 글자 “뜨거움”을 0으로 바꿔 처리하면 오류 입력이 정상 기록으로 둔갑합니다.

직접 적용. cases.json 아홉 입력 읽기 → 실행 전에 기대 상태 써 보기 → 경계 세 사례를 묶기 → 결과가 틀릴 때의 업무 영향을 설명하기

설명할 질문. 왜 28만 확인하지 않고 27.9와 28.1도 보나요?

막혔을 때. 숫자와 불리 언을 혼동하면 type(True)가 bool임을 확인하고 온도 계약과 비교합니다.

입력·답칸·확인 순서는 [실습지의 A05](#)와 [활동 원문](#)에 있습니다.

A05 · 직접 해보기

A05 · 정상·경계·오류 사례

목적

평범한 22.5 하나가 통과해도 28에서 비교 기호가 틀린 문제는 드러나지 않습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [cases.json](#), [request.md](#), [A05_input.md](#)

작업: [A05_notes.md](#)

실제 입력

27.9는 정상, 28 과 28.1은 주의입니다. 0은 유효하며 None은 미측정입니다.

단계

1. cases.json 아홉 입력 읽기
2. 실행 전에 기대 상태 써 보기

- 3. 경계 세 사례를 묶기
- 4. 결과가 틀릴 때의 업무 영향을 설명하기

예상 결과

아홉 사례의 예상과 경계 근거

확인 방법

왜 28만 확인하지 않고 27.9와 28.1도 보나요?

오류 해결

숫자와 불리 언을 혼동하면 type(True)가 bool임을 확인하고 온도 계약과 비교합니다.

힌트

검사 사례는 입력과 기대 결과를 한 쌍으로 적은 것입니다. 경계 사례는 규칙이 달라지는 바로 앞·같은 값·바로 뒤를 비교합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 왜 28만 확인하지 않고 27.9와 28.1도 보나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____
확인한 결과와 이유: _____

A06 · 검증 가능한 프롬프트

뜻과 비유. 프롬프트는 도구에게 전달하는 요청 문장입니다. 무엇을 고칠지, 어떤 값을 넣을지, 무엇을 유지할지, 어떻게 시험할지 적으면 도구의 결과를 직접 확인할 수 있습니다.

왜 필요한가. 검사 방법을 먼저 적으면 도구가 스스로 완료라고 말해도 무엇이 확인됐는지 비교할 수 있습니다.

예를 따라 읽기. “28 이상은 주의로 고치고 None은 미측정으로 남기며 아홉 사례를 실행해 결과를 보고해 주세요.”

헛갈리는 반례. “완벽한 전문가 코드로 만들어”는 어떤 결과가 맞는지 판단할 기준이 없습니다.

직접 적용. work/task.md의 여섯 칸 채우기 → 모호한 형용사 한 개를 구체 조건으로 고치기 → 짝이 요청만 보고 예상 결과 말하기 → 해석이 갈린 문장 다시 쓰기

설명할 질문. 좋은 요청에 실행 전 기대 결과가 들어가는 이유는 무엇인가요?

막혔을 때. 요청이 길어지면 목적과 유지 조건을 분리하고 이번에 필요 없는 기능은 후속 요청으로 둡니다.

입력·답칸·확인 순서는 [실습지의 A06](#)와 [활동 원문](#)에 있습니다.

A06 · 직접 해보기

A06 · 검증 가능한 프롬프트

목적

검사 방법을 먼저 적으면 도구가 스스로 완료라고 말해도 무엇이 확인됐는지 비교할 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [task.md](#), [request.md](#), [A06_input.md](#)

작업: [A06_notes.md](#), [task.md](#)

실제 입력

“28 이상은 주의로 고치고 None은 미측정으로 남기며 아홉 사례를 실행해 결과를 보고해 주세요.”

단계

1. work/task.md의 여섯 칸 채우기
2. 모호한 형용사 한 개를 구체 조건으로 고치기
3. 짝이 요청만 보고 예상 결과 말하기
4. 해석이 갈린 문장 다시 쓰기

예상 결과

입력·제약·검사 가능한 작업 요청

확인 방법

좋은 요청에 실행 전 기대 결과가 들어가는 이유는 무엇인가요?

오류 해결

요청이 길어지면 목적과 유지 조건을 분리하고 이번에 필요 없는 기능은 후속 요청으로 둡니다.

힌트

프롬프트는 도구에게 전달하는 요청 문장입니다. 무엇을 고칠지, 어떤 값을 넣을지, 무엇을 유지할지, 어떻게 시험할지 적으면 도구의 결과를 직접 확인할 수 있습니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 좋은 요청에 실행 전 기대 결과가 들어가는 이유는 무엇인가요? → _____

- 입력 파일과 내가 바꾼 위치: _____

- 실행 전 예상 결과: _____

- 실제 관찰 결과와 확인 방법: _____

- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 입력과 출력 제약

이 교시의 다른 예와 코드 해설 열기

3교시 · 변경된 코드 읽기

13:00~13:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 풀니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A07 · 변경 비교의 기호 읽기

사례로 이해하기 · 그럴듯한 완료 문장에서 근거 찾기

실무 상황

제공 응답 A는 “모든 문제를 해결했습니다”라고 말합니다. 하지만 코드에서 기본 기준을 27로 낮추고 None을 0으로 바꿨습니다. 설명이 친절하고 자신 있어도 요청한 규칙대로 고쳤는지는 따로 확인해야 합니다. 바꿨다고 말한 부분을 코드에서 찾고 실행 결과와 비교합니다.

쉬운 풀이

변경 비교는 이전과 이후의 차이를 보여 줍니다. 삭제 표시 -와 추가 표시 +는 파일 내용에 넣는 기호가 아닙니다. 주장에는 “고쳤다”, “검사했다”, “운영 가능하다”처럼 서로 다른 범위가 있습니다. 첫 주장은 코드 차이로, 둘째는 대상과 명령과 출력으로, 셋째는 실제 장비에서 실행한 결과로 확인합니다. 결과를 보지 못했다면 거짓이라고 단정하지 말고 아직 확인하지 못했다고 적습니다. 다음에 무엇을 시험할지도 정합니다.

완성 예와 풀이

code/proposal_a.py를 읽으면 `threshold=27`과 `None`을 0으로 바꾸는 줄이 보입니다. 요청의 유지 조건은 `threshold=28`이고 `None`은 미측정입니다. 먼저 두 차이를 표시한 뒤 같은 code/check_cases.py로 실행합니다. 응답의 말과 실제 함수 동작은 별도로 기록합니다. 실행 경로가 proposal_a.py인지도 확인합니다. work/classifier.py를 고쳤는데 baseline.py를 실행했다면 고친 코드를 시험한 것이 아닙니다. 복구할 때는 원본 baseline.py를 보존한 채 작업 복사본만 다시 가져옵니다.

독립 추가 복습 과제

“수정 완료, 오류 없음, 실제 장비에서도 안정적”이라는 세 주장 옆에 어떤 파일이나 실행 결과를 확인해야 하는지 적으세요. 주어진 자료가 코드 비교와 로컬 가상 검사 출력뿐일 때 어느 문장까지 보고할 수 있는지도 결정하세요.

답과 대표 오답

변경 완료에는 diff, 오류 검사는 사례별 실행 결과, 장비 안정성에는 실제 장비와 운영 조건의 관찰이 필요합니다. 현재는 가상 검사 범위만 보고하고 장비 부분은 미확인으로 남깁니다. “AI의 자신감이 높아서 승인”은 코드나 실행 결과를 확인하지 않은 답입니다. “검사 결과가 없으니 코드는 반드시 틀렸다”도 맞지 않습니다. 아직 확인하지 못한 것과 검사에서 실패한 것은 다릅니다.

확인 질문과 해설

① 코드 차이만으로 실행 성공을 아나요? 실제로 실행한 결과를 봐야 합니다. ② 통과 화면에 대상 경로가 필요한 이유는 무엇인가요? 어느 복사본을 확인했는지 식별합니다. ③ 불확실성을 어떻게 쓰나요? 관찰 사실과 아직 확인하지 않은 조건을 나누어 구체적으로 적습니다.

뜻과 비유. diff는 전후 코드에서 빠진 줄과 새 줄을 보여 주는 변경 비교입니다. -는 이전 줄, +는 새 줄이며 코드의 빠기·더하기 연산과 다릅니다.

왜 필요한가. 전체 파일을 외우지 않아도 요청이 어느 변경에 반영됐는지 확인할 수 있습니다.

예를 따라 읽기. >를 >=로 바꾼 줄은 경계 28을 포함하도록 동작을 바꿉니다.

헛갈리는 반례. 주석만 “28 이상”으로 바뀌고 실제 비교가 >이면 동작은 그대로입니다.

직접 적용. changes.diff에서 파일 이름 찾기 → 미측정·검사·경계 변경 줄 묶기 → 요청한 항목과 변경 한 곳 연결하기 → 문구 변경과 동작 변경 구분하기

설명할 질문. diff의 +와 Python의 +는 어떤 차이가 있나요?

막혔을 때. diff의 +를 원본 코드에 복사했다면 실제 소스 파일의 완성 줄을 참고해 제거합니다.

입력·답칸·확인 순서는 [실습지의 A07](#)과 [활동 원문](#)에 있습니다.

A07 · 직접 해보기

A07 · 변경 비교의 기호 읽기

목적

전체 파일을 외우지 않아도 요청이 어느 변경에 반영됐는지 확인할 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [changes.diff](#), [baseline.py](#), [reviewed.py](#), [A07_input.md](#)

작업: [A07_notes.md](#)

실제 입력

를 >=로 바꾼 줄은 경계 28을 포함하도록 동작을 바꿉니다.

단계

1. changes.diff에서 파일 이름 찾기
2. 미측정·검사·경계 변경 줄 묶기
3. 요청한 항목과 변경 한 곳 연결하기
4. 문구 변경과 동작 변경 구분하기

예상 결과

요청과 실제 변경 줄의 대응 표

확인 방법

diff의 +와 Python의 +는 어떤 차이가 있나요?

오류 해결

diff의 +를 원본 코드에 복사했다면 실제 소스 파일의 완성 줄을 참고해 제거합니다.

힌트

diff는 전후 코드에서 빠진 줄과 새 줄을 보여 주는 변경 비교입니다. -는 이전 줄, +는 새 줄이며 코드의 빼기·더하기 연산과 다릅니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: diff의 +와 Python의 +는 어떤 차이가 있나요? → ____
- 입력 파일과 내가 바꾼 위치: ____

- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 자신 있게 쓴 AI 응답 확인하기 / 작업 복사본과 되돌림

이 교시의 다른 예와 코드 해설 열기

4교시 · 예상과 실제 결과

14:00~14:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 푹니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A10 · 기준 코드의 실제 실패

사례로 이해하기 · 실패 결과를 요구와 연결하는 검사

실무 상황

프로그램이 실행되었는데 검사 결과에 FAIL이 보입니다. 비전공자는 설치가 잘못됐다고 생각하기 쉽습니다. 하지만 이 검사는 요구와 다른 출력이 나오면 의도적으로 실패 상태를 남깁니다. 실패를 읽을 수 있으면 재설치보다 수정할 규칙을 먼저 찾을 수 있습니다.

쉬운 풀이

예상은 실행 전에 규칙으로 정한 답이고 실제는 실행으로 관찰한 답입니다. 종료 코드 0은 이 검사에서 모든 사례가 맞았다는 뜻이며 1은 적어도 하나가 달랐다는 뜻입니다. 이 의미는 프로그램마다 다릅니다. TypeError가 출력에 보이면 값 종류에 맞지 않는 비교가 발생했을 수 있습니다. 누락과 종류와 범위를 먼저 확인한 뒤 경계를 비교하면 오류 상황을 정상 온도로 착각하지 않습니다.

완성 예와 풀이

압축을 푼 course에서 `python code/check_cases.py code/baseline.py`를 실행합니다. baseline은 28을 정상, None을 정상으로 반환하고 문자 비교에서는 예외가 발생합니다. 따라서 성공적인 수업 시연에서도 종료 코드가 1입니다. `reviewed.py`는 None을 먼저 미측정으로 처리하고 정확한 숫자 종류와 0~50 범위를 확인한 뒤 `>=28`을 적용합니다. `python code/check_cases.py code/reviewed.py`는 제공 아홉 사례에서 9/9입니다. 예상 목록이 같고 실행 대상만 달라졌음을 확인해야 변화의 원인을 설명할 수 있습니다.

독립 추가 복습 과제

입력 0, None, True, "28", 50, 51의 출력을 먼저 표로 작성하세요. 이후 제공 완성 함수의 조건을 위에서 아래로 따라가며 각 입력이 어느 줄에서 반환되는지 설명하세요. 조건 순서를 바꾸면 문자 비교가 어떻게 달라지는지도 판단하세요.

답과 대표 오답

0은 정상, None은 미측정, True와 "28"과 51은 입력 오류, 50은 주의입니다. "True는 1과 같으니 정상"은 오늘의 계약이 허용한 정확한 종류를 놓쳤습니다. "None은 0"은 값 없음과 유효한 영점을 혼동합니다.

확인 질문과 해설

① FAIL이면 파이썬이 실행되지 않았나요? 실행된 뒤 기대와 달랐을 수 있습니다. ② 기준만 낮추면 해결되나요? 기존 정상 경계를 깨뜨립니다. ③ 아홉 통과로 모든 입력을 보장하나요? 제공된 검사 범위만 확인한 것이므로 새로운 입력 조건은 별도로 검토합니다.

뜻과 비유. 실패 검사는 실수로 망가진 수업이 아니라 요구와 현재 동작의 차이를 드러내는 관찰입니다. 검사 프로그램은 기대와 실재를 나란히 비교합니다.

왜 필요한가. 수정 전 실패를 재현해 두면 수정 후 무엇이 달라졌는지 확인할 기준이 생깁니다.

예를 따라 읽기. python code/check_cases.py code/baseline.py를 실행하면 경계와 미측정 등의 차이가 FAIL로 나옵니다.

헛갈리는 반례. 종료 코드 1을 보고 무조건 재설치하지 않습니다. 이 검사기는 실패 사례가 있으면 의도적으로 1을 반환합니다.

직접 적용. 전체 검사 코드와 줄 해설 읽기 → 원본 실행 명령 확인하기 → 아홉 줄 결과에서 FAIL의 입력 찾기 → 예상과 실제 차이를 표에 남기기

설명할 질문. 테스트가 실패한 것과 실행환경이 고장 난 것은 어떻게 구별하나요?

막혔을 때. 모듈 경로 오류면 course 폴더에서 명령을 실행하고 파일 경로 인수를 빠뜨리지 않았는지 봅니다.

입력·답칸·확인 순서는 [실습지의 A10](#)와 [활동 원문](#)에 있습니다.

A10 · 직접 해보기

A10 · 기준 코드의 실제 실패

목적

수정 전 실패를 재현해 두면 수정 후 무엇이 달라졌는지 확인할 기준이 생깁니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [check_cases.py](#), [baseline.py](#), [cases.json](#), [A10_input.md](#)

작업: [A10_notes.md](#)

실제 입력

python code/check_cases.py code/baseline.py를 실행하면 경계와 미측정 등의 차이가 FAIL로 나옵니다.

단계

1. 전체 검사 코드와 줄 해설 읽기
2. 원본 실행 명령 확인하기
3. 아홉 줄 결과에서 FAIL의 입력 찾기
4. 예상과 실제 차이를 표에 남기기

예상 결과

기준 코드의 실제 결과와 실패 원인

확인 방법

테스트가 실패한 것과 실행환경이 고장 난 것은 어떻게 구별하나요?

오류 해결

모듈 경로 오류면 course 폴더에서 명령을 실행하고 파일 경로 인수를 빠뜨리지 않았는지 봅니다.

힌트

실패 검사는 실수로 망가진 수업이 아니라 요구와 현재 동작의 차이를 드러내는 관찰입니다. 검사 프로그램은 기대와 실제를 나란히 비교합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 테스트가 실패한 것과 실행환경이 고장 난 것은 어떻게 구별하나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

A11 · 경계 비교 기호 수정

뜻과 비유. >는 기준보다 큰 값, >=는 기준과 같은 값까지 포함합니다. 말로 쓴 “이상”과 코드 비교 기호를 연결합니다.

왜 필요한가. 경계 한 글자는 업무 분류를 바꿉니다. 정상 값만 시험하면 이 차이를 놓칩니다.

예를 따라 읽기. 작업본의 value > threshold를 value >= threshold로 바꾸면 28의 결과가 정상에서 주의로 달라집니다.

헛갈리는 반례. 기본 threshold를 27로 내리는 것은 같은 수정이 아닙니다. 27.9까지 주의가 되어 규칙을 바꿉니다.

직접 적용. 수정 전 28결과 기록하기 → work의 비교 기호 한곳 수정하기 → 저장 후 같은 검사 다시 실행하기 → 27.9·28·28.1 변화 비교하기

설명할 질문. 기준값을 바꾸는 것과 비교 기호를 바꾸는 것은 왜 다른가요?

막혔을 때. 결과가 그대로면 저장 여부와 실행 경로를 먼저 확인합니다.

입력·답칸·확인 순서는 [실습지의 A11](#)와 [활동 원문](#)에 있습니다.

A11 · 직접 해보기

A11 · 경계 비교 기호 수정

목적

경계 한 글자는 업무 분류를 바꿉니다. 정상 값만 시험하면 이 차이를 놓칩니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [classifier.py](#), [cases.json](#), [A11_input.md](#)

작업: [A11_notes.md](#), [classifier.py](#)

실제 입력

작업본의 value > threshold를 value >= threshold로 바꾸면 28의 결과가 정상에서 주의로 달라집니다.

단계

1. 수정 전 28결과 기록하기
2. work의 비교 기호 한곳 수정하기
3. 저장 후 같은 검사 다시 실행하기
4. 27.9·28·28.1 변화 비교하기

예상 결과

경계 수정 전후의 세 입력 결과

확인 방법

기준값을 바꾸는 것과 비교 기호를 바꾸는 것은 왜 다른가요?

오류 해결

결과가 그대로면 저장 여부와 실행 경로를 먼저 확인합니다.

힌트

는 기준보다 큰 값, >=는 기준과 같은 값까지 포함합니다. 말로 쓴 "이상"과 코드 비교 기호를 연결합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 기준값을 바꾸는 것과 비교 기호를 바꾸는 것은 왜 다른가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 누락과 잘못된 입력 처리

이 교시의 다른 예와 코드 해설 열기

5교시 · 오류 재현과 수정

15:00~15:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 품니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A15 · 회귀 확인과 증거

뜻과 비유. 회귀는 수정한 뒤 예전에 맞던 동작이 다시 틀리는 현상입니다. 수정한 사례 뿐 아니라 관련된 기존 사례도 함께 확인합니다.

왜 필요한가. 경계를 고치며 기준을 27로 바꾸면 28은 맞아도 27.9가 틀려집니다.

예를 따라 읽기. baseline.proposal_a-reviewed의 아홉 결과를 같은 기대 값으로 비교합니다.

햇갈리는 반례. 검사를 새 코드에 유리하게 바꾸거나 통과 한 줄만 보고하면 근거가 사라집니다.

직접 적용. 세 후보에 같은 검사 실행하기 → 통과 수와 실패 입력 표로 비교하기 → 요구 위반과 실패의 연결 쓰기 → 원본 검사 파일이 유지됐는지 확인하기

설명할 질문. 통과 수만 같으면 같은 품질이라고 할 수 있나요?

막혔을 때. FAIL 수가 줄어도 새로운 실패가 입력되었는지 각 행을 확인합니다.

입력·답칸·확인 순서는 [실습지의 A15](#)와 [활동 원문](#)에 있습니다.

A15 · 직접 해보기

A15 · 회귀 확인과 증거

목적

경계를 고치며 기준을 27로 바꾸면 28 은 맞아도 27.9가 틀려집니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [baseline.py](#), [proposal_a.py](#), [reviewed.py](#), [check_cases.py](#), [A15_input.md](#)

작업: [A15_notes.md](#)

실제 입력

baseline-proposal_a-reviewed의 아홉 결과를 같은 기대 값으로 비교합니다.

단계

1. 세 후보에 같은 검사 실행하기
2. 통과 수와 실패 입력 표로 비교하기
3. 요구 위반과 실패의 연결 쓰기
4. 원본 검사 파일이 유지됐는지 확인하기

예상 결과

세 후보의 비교표와 선택 근거

확인 방법

통과 수만 같으면 같은 품질이라고 할 수 있나요?

오류 해결

FAIL 수가 줄어도 새로운 실패가 입력되었는지 각 행을 확인합니다.

힌트

회귀는 수정한 뒤 예전에 맞던 동작이 다시 틀리는 현상입니다. 수정한 사례 뿐 아니라 관련된 기존 사례도 함께 확인합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 통과 수만 같으면 같은 품질이라고 할 수 있나요? → _____

- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____
확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 재현 가능한 오류 보고 / 수정 요청을 줄이기

이 교시의 다른 예와 코드 해설 열기

6교시 · 새 요청과 인수인계

16:00~16:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

이 교시에는 50분 동안 아래 활동을 진행합니다. 수업 활동부터 하고 남은 문제는 복습으로 품니다. 학생에게 어느 줄을 고쳤고 결과가 어떻게 달라졌는지 물어봅니다.

A16 · 새 조건의 영향 분석

사례로 이해하기 · 새 기준과 완료 범위를 인계하기

실무 상황

담당자가 새 장소에는 기준 30을 적용해 달라고 승인했습니다. 기존 모든 장소의 기본값을 30으로 바꾸면 이전 업무가 달라 집니다. 새 요청이 특정 장소만의 조건인지 전체 기본값 변경인지 먼저 확인하고 영향 받는 호출과 검사를 찾습니다.

쉬운 풀이

함수 인수는 호출할 때 전달하는 입력입니다. 기본값은 그 인수를 생략했을 때 사용하는 값입니다. `classify(29.9, threshold=30)`과 `classify(29.9)`는 같은 입력 온도지만 기준이 다르므로 결과가 다를 수 있습니다. 요구 변경은 검사의 기대를 바꿀 수 있으나 어떤 승인된 요구에 따른 변경인지 근거를 남겨야 합니다. 기존 실패를 숨기려고 기대를 바꾼 상황과 구별합니다.

완성 예와 풀이

새 장소 요청서에 기준 30, 기본 계약 28 유지, 유효 범위 0~50 유지라고 씁니다. 새 호출의 29.9는 정상, 30과 30.1은 주의입니다. 기본 호출의 28은 여전히 주의입니다. `None`은 두 호출 모두 미측정이고 범위 밖은 입력 오류입니다. 인계에는 수정한 파일, 변경 조건, 기본 아홉 사례 결과, 새 경계 세 사례 결과, 실제 AI 호출 여부를 씁니다. 오늘은 제공 응답을 검토한 오프라인 활동이며 실제 센서 안전 판단과 실제 AI 서비스 연결은 확인하지 않았다고 기록합니다.

독립 추가 복습 과제

기준 30을 특정 장소에만 적용하는 요청과 전체 기본값을 30으로 바꾸는 요청을 각각 한 문장으로 작성하세요. 두 요청에서 기대가 달라질 기본 호출 한 개를 고르고, 동료가 다시 확인할 명령과 자료 위치를 인계문에 넣으세요.

답과 대표 오답

특정 장소에는 threshold 인수를 명시하고 전체 변경에는 기본값 변경 승인을 명시합니다. 기본 classify(28)은 전자에서 주의, 후자에서 정상입니다. "새 검사가 통과했으니 모든 서비스 운영 완료"는 검증 범위를 넘었습니다. "새 요청은 절대 기대를 바꾸면 안 된다"도 승인된 요구 변경을 무시합니다.

확인 질문과 해설

① 기본값과 인수는 왜 구별하나요? 영향 받는 호출 범위가 다릅니다. ② 동료가 같은 결과를 보려면 무엇이 필요한가요? 대상 파일·실행 위치·입력·기대를 연결한 기록입니다. ③ 아직 모르는 사항은 어떻게 전달하나요? 실제 환경, 장비, 계정 등 미확인 항목과 다음 확인 담당을 구체적으로 남깁니다.

뜻과 비유. 새 업무 조건은 기존 규칙과 독립인지 먼저 확인한 뒤 작은 사례로 설계합니다. 오늘은 추가 코드보다 변경이 영향 줄 입력을 설명하는 일에 집중합니다.

왜 필요한가. 특수 기준을 추가하면 조건 순서와 기존 계약을 함께 검토해야 합니다.

예를 따라 읽기. 담당자가 기준을 30으로 바꿔 달라고 새로 요청했다면 29.9·30·30.1을 새 경계로 예상합니다.

헛갈리는 반례. 승인된 새 요청 없이 기준을 바꾸는 것과 업무 담당자가 조건을 변경한 것은 다릅니다.

직접 적용. 기준 30이라는 새 요청서 작성하기 → 영향 받을 기대 값 찾기 → 29.9·30·30.1의 예상 작성하기 → 기본 기준 28 을 유지할지 인수로 전달할지 설명하기

설명할 질문. 새 조건에서 어떤 기준 검사를 그대로 써야 하나요?

막혔을 때. 기본 기준 변경과 함수 인수 전달을 혼동하면 classify(30,threshold =30)호출 예를 읽습니다.

입력·답칸·확인 순서는 [실습지의 A16](#)과 [활동 원문](#)에 있습니다.

A16 · 직접 해보기

A16 · 새 조건의 영향 분석

목적

특수 기준을 추가하면 조건 순서와 기존 계약을 함께 검토해야 합니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [reviewed.py](#), [task.md](#), [A16_input.md](#)

작업: [A16_notes.md](#), [task.md](#)

실제 입력

담당자가 기준을 30으로 바꿔 달라고 새로 요청했다면 29.9·30·30.1을 새 경계로 예상합니다.

단계

1. 기준 30이라는 새 요청서 작성하기

2. 영향 받을 기대 값 찾기
3. 29.9-30-30.1의 예상 작성하기
4. 기본 기준 28 을 유지할지 인수로 전달할지 설명하기

예상 결과

추가 조건의 영향 표와 검증 계획

확인 방법

새 조건에서 어떤 기존 검사를 그대로 써야하나요?

오류 해결

기본 기준 변경과 함수 인수 전달을 혼동하면 `classify(30,threshold =30)`호출 예를 읽습니다.

힌트

새 업무 조건은 기존 규칙과 독립인지 먼저 확인한 뒤 작은 사례로 설계합니다. 오늘은 추가 코드보다 변경이 영향 줄 입력을 설명하는 일에 집중합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 새 조건에서 어떤 기존 검사를 그대로 써야하나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

A18 · 완료 범위와 다음 확인

뜻과 비유. 인수인계에서는 요청받은 일, 고친 파일, 실행 결과, 아직 시험하지 못한 조건을 다음 사람에게 알려 줍니다.

왜 필요한가. 합성 데이터 검사와 실제 센서 운영은 다른 범위입니다. 확인하지 않은 서비스 호출이나 장비 연결을 완료라고 쓰지 않습니다.

예를 따라 읽기. 분류기 아홉 사례 통과·오프라인 응답 검토·실제 AI 호출 미수행을 함께 적습니다.

헛갈리는 반례. AI가 코드를 썼다고 가정하거나 검사 통과만으로 실제 안전 시스템 완성이라고 표현하지 않습니다.

직접 적용. 세 문장 인계 작성하기 → 실제 AI 호출 여부 명시하기 → 미검증 조건 두 개 쓰기 → 0과 None의 차이, 기준과 경계값의 차이, AI 설명을 확인하는 방법에 답하기

설명할 질문. 자료 작성과 실제 운영 검증을 어떻게 구분하나요?

막혔을 때. 완료 문장이 추상적이면 파일·입력 수·실행 방법을 넣습니다.

입력·답칸·확인 순서는 [실습지의 A18](#)와 [활동 원문](#)에 있습니다.

공식 근거와 원본

확인일2026-09-26. 정의는 짧게 요약하고 활동·데이터·설명은 수업용으로 새로 작성했습니다.

- [코드에이전트의 작업 흐름](#): 읽기·변경·실행·검토의 개념 참고, 특정 계정 사용 실습 아님
- [Python 조건과 함수](#): 함수·조건·반환 기본 동작

개념 보충 공식 근거 (2026-10-05 확인)

- [GitHub 코드에이전트의 도구 작업과 검토](#)

A18 · 직접 해보기

A18 · 완료 범위와 다음 확인

목적

합성 데이터 검사와 실제 센서 운영은 다른 범위입니다. 확인하지 않은 서비스 호출이나 장비 연결을 완료라고 쓰지 않습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [review.md](#), [A18_input.md](#)

작업: [A18_notes.md](#), [review.md](#)

실제 입력

분류기 아홉 사례 통과·오프라인 응답 검토·실제 AI 호출 미수행을 함께 적습니다.

단계

1. 세 문장 인계 작성하기
2. 실제 AI 호출 여부 명시하기
3. 미검증 조건 두 개 쓰기
4. 0과 None의 차이, 기준과 경계값의 차이, AI 설명을 확인하는 방법에 답하기

예상 결과

고친 파일, 실행 결과, 아직 시험하지 못한 조건을 적은 안내

확인 방법

자료 작성과 실제 운영 검증을 어떻게 구분하나요?

오류 해결

완료 문장이 추상적이면 파일·입력 수·실행 방법을 넣습니다.

힌트

인수인계에서는 요청받은 일, 고친 파일, 실행 결과, 아직 시험하지 못한 조건을 다음 사람에게 알려 줍니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 자료 작성과 실제 운영 검증을 어떻게 구분하나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 동료의 변경을 설명하기

[이 교사의 다른 예와 코드 해설 열기](#)

오늘 배운 내용 확인

확인할 것	배점	충족 기준
배운 내용 설명	20점	사용한 개념과 선택한 이유를 예로 설명한다.
결과물 완성	40점	오늘 필수 활동의 파일·표·기획 내용을 빠짐없이 완성한다.
결과 확인	30점	예상과 결과를 비교하고 다른 입력 한 가지로 다시 확인한다.

저장과 전달	10점	파일 이름과 실행·읽는 방법을 함께 적어 다른 사람이 확인할 수 있다.
--------	-----	---

각 항목: 모두 충족하면 배점 전부, 일부 충족하면 배점의 절반, 근거가 없으면 0점. 오류를 발견하고 고친 기록도 확인 근거로 사용합니다.

남길 결과: work/A16_notes.md / work/A18_notes.md

세 가지 확인 질문

1. AI가 코드를 고쳤다고 답하면 어떤 파일과 실행 결과를 확인해야 하나요?

내 설명: _____
2. 오늘 필수 활동에서 값을 한 곳 바꾼 예를 고르세요. 바꾸기 전 예상과 확인한 결과를 설명해 보세요.

내 설명: _____
3. 마지막 결과물의 파일 이름과 읽거나 실행하는 방법을 적어 보세요. 다른 사람이 확인할 수 있도록 한 문장씩 안내합니다.

내 설명: _____